

NumCompute Stream Framework

June 5, 2026

1 Introduction

This report provides the details for the design and implementation of **NumCompute Stream**, an extended version of the NumCompute library. The framework is designed for streaming machine learning, supporting incremental updates, ensemble methods, and real-time visualisation, built entirely from scratch using plain-Python and NumPy.

2 Design Decisions

2.1 Streaming Architecture

The core design was to ensure that every component in the machine learning pipeline, from preprocessing to evaluation, could handle data incrementally.

- **Partial Fit API:** Adoption of `.partial_fit()` convention for all transformers (`StandardScaler`, `SimpleImputer`, etc) and classifiers. This allows the system to update its internal state (like running means or tree structures) without retraining on the entire historical dataset, making the training simpler.
- **StreamTrainer:** Implementation of a high-level manager, `StreamTrainer`, to orchestrate the Pre-quential Evaluation loop, which is the standard for evaluating streaming algorithms.

2.2 Model Ensembling

The `RandomForestClassifier` was implemented using a bagging approach.

- **Bootstrapping in Streams:** For each data chunk, the ensemble creates bootstrap samples to update individual `DecisionTreeClassifier` instances.
- **Majority Voting:** Prediction is handled via a majority vote across all base estimators, ensuring robustness against noise in individual chunks.

2.3 Numerical Stability

To ensure the framework is production-ready, we implemented:

- **Welford's Algorithm:** Used for incremental variance calculation to prevent catastrophic cancellation.
- **NaN Handling:** `SimpleImputer` and core statistical functions were redesigned to handle missing values gracefully using NumPy's vectorized operations.

Category	Edge Cases Handled
Preprocessing	Zero-variance columns, feature count mismatches, empty chunks.
Metrics	Division by zero in F1/Precision, single-class ROC curves.
Pipelines	Duplicate step names, missing required methods, validation errors.
Trees	Maximum depth limits, tie resolution in Gini/Entropy splits.

Table 1: Summary of handled edge cases.

3 Testing & Edge Cases

The framework was validated using a suite of over 119 unit tests. Key focus areas included.

4 Benchmark Results

Benchmarks were conducted comparing the streaming performance of a single Decision Tree against the Random Forest ensemble on the Iris dataset.

Metric	Single Decision Tree	Random Forest (10 Trees)
Initial Accuracy	0.00	0.00
Final Cumulative Accuracy	0.8533	0.8200

Table 2: Streaming performance comparison.

The results demonstrate that due to the small dataset available and Decision Tree’s higher stability in the learning phases initially, it had a greater cumulative accuracy. While both models ended with the same high chunk accuracy (0.9333), the Decision Tree was more consistently correct throughout the entire process.

5 Reflections

Building a streaming-compatible framework from scratch highlighted the complexities of state management in online learning. Different to batch learning, where the entire dataset is available, streaming requires careful accumulation and monitoring of statistics and incremental model growth. The transition from a static to a dynamic pipeline demonstrated a deep understanding of algorithmic stability and NumPy’s vectorised operations to maintain performance without external libraries. Future work could include implementing Hoeffding Trees for more efficient incremental growth.